MPECI 2025/2026 — Practical Guide 3

Exam Study Notes: Markov Chains & PageRank

1. THE CRITICAL CONVENTION — Read This First

This guide uses a specific matrix convention that differs from how transition tables are typically written. Getting this wrong invalidates every calculation.

Convention: t_ij = P(transition FROM state j TO state i)

	Written table (rows=from, cols=to)	Matrix T in this guide
Entry meaning	row i, col j = P(from i, to j)	t_ij = P(from j, to i)
Column property	rows sum to 1 (row-stochastic)	COLUMNS sum to 1 (col-stochastic)
Relationship	—	T = transpose of the table
State update	—	x_{n+1} = T * x_n
After n steps	—	x_n = T^n * x_0

■ EXAM TRAP: The weather table in Q1 has rows=current state, cols=next state. To build T, you TRANSPOSE it. T(i,j) = table(j,i).

```
% Verify T is column-stochastic: sum(T) % must give all 1s (column sums) sum(T, 1) % same as sum(T) — sums down each column
```

✓ Quick check: sum(T) should return a row vector of all 1s. If rows sum to 1 instead, your T is transposed.

2. MARKOV CHAIN FUNDAMENTALS

Markov Property

The future state depends ONLY on the current state, not on the history. Formally: P(X_{n+1}=j | X_n=i, X_{n-1}=...) = P(X_{n+1}=j | X_n=i)

State Vector x

x is a column vector where x_i = P(currently in state i). It must satisfy: x_i >= 0 for all i, and sum(x) = 1.

```
x_0 = initial distribution (e.g. [1;0;0] if starting in state 1)
```

```
x_n = T^n * x_0 (distribution after n steps)
```

Multi-step Transitions — Key Patterns

Task	Formula	MATLAB
P(state j after n steps | start i)	(T^n)(j, i) — element [j,i] of T^n	Tn = T^n; Tn(j,i)
P(in state s on day k | start x)	x_k = T^(k-1) * x_0; answer = x_k(s)	x = T^(k-1) * x0; x(s)
P(day k=s AND day k+1=s | start x)	P(day k=s) * T(s,s)	x(s) * T(s,s)
Expected days in state s (n days)	sum over k=1..n of x_k(s)	loop: x=T*x; total=total+x(s)
Stationary distribution pi	T*pi = pi, sum(pi)=1	eig(T) — eigvec for eigenval=1

Computing Expected Days (Q1 pattern)

```
% Expected days in each state over n_days, starting from x0: total = zeros(size(x0)); x = x0; for day = 1:n_days total = total + x; % accumulate probability x = T * x; % advance one step end % total(i) = expected number of days in state i
```

■ Include day 1 (the starting day) in the loop — start accumulating BEFORE the first T*x update, since day 1 is the known starting state.

Expected Value with a Reward Function (Q1f pattern)

If there is a reward r(s) associated with being in state s (e.g. P(pain|weather)), the expected total reward over n days is:

$$E[\text{total reward}] = \sum_{k=1}^{n} r' * x_k$$

```
% r = reward vector, e.g. [0.1; 0.3; 0.5] for pain probabilities total_reward = 0; x = x0; for day = 1:n_days total_reward = total_reward + r' * x; x = T * x; end
```

Stationary Distribution

The stationary (limiting) distribution pi is the state the chain converges to after many steps, regardless of the starting state (for irreducible, aperiodic chains).

$$T * pi = pi \text{ and } sum(pi) = 1$$

```
% Method 1: eigenvector (recommended) [V, D] = eig(T); [~, idx] = min(abs(diag(D) - 1)); % eigenvalue closest to 1 pi = abs(V(:, idx)); pi = pi / sum(pi); % normalise % Method 2: power iteration (run many steps) pi = T^1000 * x0; % converges for large n
```

✓ Convergence check: if T^100 and T^200 give the same result, you have reached the stationary distribution.

3. GENERATING A RANDOM STOCHASTIC MATRIX (Q4)

To generate a random column-stochastic matrix: generate random entries, then normalise each column so it sums to 1.

```
R = rand(n, n); T = R ./ sum(R, 1); % divide each element by its column sum % Verify: sum(T) % should be all 1s max(abs(sum(T)-1)) % should be ~0 (floating point)
```

■ sum(R, 1) sums along dimension 1 (down columns). The ./ broadcasts correctly so each column is divided by its own sum.

4. PAGERANK — CONCEPTS & ALGORITHMS

Hyperlink Matrix H

H(i,j) = 1 / (number of outlinks from page j), if page j links to page i. Each column sums to 1 — unless j is a dead-end (no outlinks).

```
r_{n+1} = H * r_n basic PageRank iteration
```

Two Problems with Basic PageRank

Problem	Definition	Effect	How to detect
Dead-end	Page with NO outgoing links	Column of H is all zeros; rank leaks out of the system	sum(H(:,j)) == 0 for some j
Spider trap	Group of pages that link ONLY to each other (closed loop)	All rank eventually accumulates in the trap pages; they get 1	Subset of pages with no links to the rest of the g

Fix 1 — Dead-end only: replace zero columns with uniform 1/n

```
H_fixed = H; for j = 1:n if sum(H(:,j)) == 0 % dead-end detected H_fixed(:,j) = 1/n; % teleport uniformly end end % Now all columns sum to 1
```

Fix 2 — Both problems: Google Matrix A (with teleportation factor beta)

```
A = beta * H_fixed + (1 - beta) * (1/n) * ones(n,n)
```

Term	Meaning	Typical value
beta	Probability of following a real link (damping factor)	0.8 or 0.85
1-beta	Probability of teleporting to any page uniformly	0.15 or 0.2
(1/n)*ones(n,n)	Teleportation matrix: uniform probability to all n pages	—
A	Google matrix — always column-stochastic, irreducible, aperiodic	

■ beta fixes BOTH problems: the (1-beta) teleportation term prevents rank from being trapped in spider traps OR lost at dead-ends. Always apply Fix 1 (dead-end columns) BEFORE building A.

PageRank Iteration — Full Pipeline

```
% Step 1: build H from link diagram % Step 2: fix dead-ends for j=1:n; if sum(H(:,j))==0; H(:,j)=1/n; end; end % Step 3: build Google matrix A = beta*H + (1-beta)*(1/n)*ones(n); % Step 4a: fixed iterations r = ones(n,1)/n; % uniform start for iter = 1:40 r = A * r; end % Step 4b: convergence criterion r = ones(n,1)/n; iter = 0; while true r_old = r; r = A*r; iter = iter+1; if max(abs(r-r_old)) < 1e-4; break; end end
```

Non-Iterative PageRank (eigenvector method)

The stationary distribution of A IS the PageRank vector. Solve A*pi = pi directly using eigendecomposition:

```
[V, D] = eig(A); [~, idx] = min(abs(diag(D) - 1)); % eigenvalue = 1 pi = abs(V(:, idx)); pi = pi / sum(pi);
```

✓ The iterative and eigenvector methods should give identical results. Use iterative in practice (faster for large n), eigenvector to verify.

5. WORKED EXAMPLE — Q1 WEATHER CHAIN

Building T from the table

Table given (row=today, col=tomorrow):

Today \ Tomorrow	Sunny	Cloudy	Rainy
Sunny	0.7	0.2	0.1
Cloudy	0.2	0.3	0.5
Rainy	0.3	0.3	0.4

Each ROW of the table becomes a COLUMN of T (because T(i,j) = P(to i | from j)):

```
T = [0.7 0.2 0.3; % col1=fromSunny, col2=fromCloudy, col3=fromRainy 0.2 0.3 0.3; 0.1 0.5 0.4];
```

Q1(b) — P(day2=S AND day3=S | day1=S)

Both days must be sunny. Since transitions are independent step-by-step:

```
P = T(1,1) * T(1,1) = 0.7 * 0.7 = 0.49
```

Q1(c) — P(day6=S AND day7=S | day1=R) [Jan 1=Wed, weekend=day 6,7]

```
x5 = T^5 * [0;0;1]; % distribution on day 6 (5 steps from day 1) P = x5(1) * T(1,1); % P(day6=S) * P(day7=S | day6=S)
```

■ 5 steps from day 1 gives the distribution on day 6, not day 5. n steps forward from day 1 = day n+1.

Q1(f) — Expected pain days

```
r = [0.1; 0.3; 0.5]; % P(pain | Sunny/Cloudy/Rainy) pain = 0; x = x0; for day = 1:31 pain = pain + r' * x; x = T * x; end
```

6. PAGERANK EXAMPLE — Q5 WEB GRAPH

Link structure (A=1,B=2,C=3,D=4,E=5,F=6)

Page	Outgoing links	Col of H	Notes
A	F	H(6,1)=1	Only 1 outlink
B	A, E	H(1,2)=H(5,2)=0.5	2 outlinks
C	B, D	H(2,3)=H(4,3)=0.5	2 outlinks
D	C	H(3,4)=1	Spider trap with C
E	B, F	H(2,5)=H(6,5)=0.5	2 outlinks
F	(none)	col 6 = zeros	DEAD-END

Spider trap: C and D link only to each other (C→D→C). All PageRank eventually drains here.

Dead-end: F has no outlinks. Its column in H is all zeros, causing probability mass to disappear each iteration.

Version	What is fixed	Expected effect on PageRank
Basic H	Nothing	Rank leaks (F) and pools in C/D trap
H_fixed (dead-end)	F column set to 1/6 uniform	No more leakage; C/D trap still dominates
Google matrix A (beta=0.8)	Both dead-end AND spider-trap	Balanced PageRank; C or D likely highest

7. QUICK REFERENCE — MATLAB COMMANDS

Command	Purpose

T^n	Matrix power — n-step transition matrix
T^n * x0	State distribution after n steps
sum(T)	Column sums (verify = all 1s for col-stochastic)
R ./ sum(R,1)	Normalise columns of R to sum to 1
eig(T)	Eigenvalues and eigenvectors of T
[V,D]=eig(T); abs(V(:,idx))	Stationary distribution (eigvec for eigenval=1)
max(abs(r_new - r_old))	Convergence check: max absolute change
[~,idx]=min(abs(diag(D)-1))	Find index of eigenvalue closest to 1
ones(n,n)/n	Uniform teleportation matrix (all entries = 1/n)
A = b*H + (1-b)/n*ones(n)	Google matrix with damping factor b (beta)

8. EXAM TIPS & COMMON MISTAKES

• **Convention is everything:** Always verify sum(T) gives all 1s (column sums). If sum(T,2) gives all 1s instead, your T is transposed and every result will be wrong.

• **Table to matrix:** When a table is given row=from, col=to, the matrix T is its TRANSPOSE. Row i of the table becomes column i of T.

• **Day indexing:** T^n * x0 gives the distribution n steps AFTER the start. If day 1 is the starting day, day k requires T^(k-1) * x0. Jan 1=Wed means the first weekend (Sat=day6, Sun=day7) needs T^5 and T^6.

• **Expected days loop:** Accumulate x BEFORE updating (total += x; x = T*x), not after. This includes the starting day in the count.

• **Stationary distribution:** Convergence to stationarity means the chain 'forgets' its starting state. At n=100+, starting sunny vs rainy gives nearly identical results — this is the conclusion expected in Q1(e).

• **Dead-end vs spider trap:** Dead-end = a page with zero outlinks (column of zeros). Spider trap = a closed group of pages that never links outside. They require different fixes: dead-end fix first, then Google matrix for both.

• **Google matrix build order:** (1) Build H, (2) fix dead-end columns to 1/n, (3) apply A = beta*H_fixed + (1-beta)*(1/n)*ones(n). Skipping step 2 means the teleportation term does not fully compensate.

• **Random stochastic matrix:** R=rand(n,n); T=R./sum(R,1). The ./ with sum(R,1) divides each column by its sum. Do NOT use sum(R) without the dimension argument — same result here but ambiguous.

• **Non-iterative PageRank:** The PageRank vector is the eigenvector of A corresponding to eigenvalue 1. Use abs() on the eigenvector before normalising since MATLAB may return complex values due to numerical precision.

• **Convergence conclusion:** For any ergodic Markov chain (irreducible + aperiodic), the distribution converges to a unique stationary distribution pi, regardless of the starting state. The Google matrix guarantees this.